ПЗ №7. Изучение формата Markdown

Цель и задачи занятия

Основная цель: освоить синтаксис Markdown и научиться оформлять техническую документацию в этом формате — как основу подхода Doc as Code.

Задачи:

- изучить базовый синтаксис Markdown (заголовки, форматирование текста, списки, ссылки, изображения, таблицы, код);
- научиться работать в онлайн-редакторе StackEdit с предпросмотром;
- отработать навык оформления технического документа в Markdown;
- подготовить основу для ведения документации в репозитории (связь с лекцией о Doc as Code).

Теоретический минимум

1. Что такое Markdown и зачем он нужен

Markdown — это лёгкий язык разметки, который позволяет оформлять текст с помощью простых символов (решёток, звёздочек, дефисов) и при этом оставаться **читаемым в исходном виде**.

Главная идея: обычный текст + минимальная разметка = документ, который можно:

- читать как есть (без рендеринга);
- легко сравнивать в Git (видно, что изменилось построчно);
- конвертировать в HTML, PDF, DOCX;
- публиковать на GitHub, в вики, на сайтах документации.

Зачем Markdown веб-разработчику:

- это стандарт оформления `README.md` на GitHub;
- основа подхода **Doc as Code**;
- используется в документации API;
- применяется в Jira, Trello, Notion;
- на нём пишутся статьи, инструкции, комментарии.

Файлы Markdown имеют расширение `.md`.

---

2. Знакомство со StackEdit

StackEdit (https://stackedit.io/) — бесплатный онлайн-редактор Markdown, который работает в браузере. Не требует установки и регистрации.

Как устроен интерфейс:

- **Слева** — панель файлов и меню.
- **В центре** — область редактирования (сюда вы набираете текст).
- **Справа** — предпросмотр (здесь сразу видно результат).

Основные действия:

- **Создать новый файл:** меню слева → иконка «+» → «New file».
- **Переименовать файл:** клик по названию файла в левой панели.
- **Экспорт:** меню (три точки вверху справа) → «Export» → «Export as Markdown» (скачать `.md`) или «Export as HTML» / «PDF».
- **Сохранение:** StackEdit хранит файлы в браузере автоматически. Для надёжности используйте экспорт в `.md`.

> **Важно:** работайте **только** в StackEdit. Все задания выполняются в этом редакторе.

---

3. Базовый синтаксис Markdown (краткий справочник)

Элемент	Синтаксис	Пример результата
Заголовок 1	`# Текст`	Крупный заголовок
Заголовок 2	`## Текст`	Подзаголовок
Заголовок 3	`### Текст`	Под-подзаголовок
Жирный	`**текст**`	**текст**
Курсив	`*текст*`	*текст*
Зачёркнутый	`~~текст~~`	~~текст~~
Код в строке	`` `код` ``	`код`
Маркированный список	`- пункт`	• пункт
Нумерованный список	`1. пункт`	1. пункт
Чек-лист	`- [x] готово`	☑ готово

Элемент	Синтаксис	Пример результата
Ссылка	`[текст](url)`	текст (кликабельно)
Изображение	`![alt](url)`	картинка
Цитата	`> текст`	цитата
Блок кода	```` ```js ... ``` ````	блок с подсветкой
Таблица	`| a | b |` + строка `|---|---|`	таблица
Линия	`---`	горизонтальная линия
Экранирование	`\*текст\*`	*текст* (без курсива)

Материалы и оборудование

- ПК с доступом в интернет и браузером.
- **StackEdit** (https://stackedit.io/).
- Шпаргалка по Markdown.

Задание

Выполнить **четыре упражнения** (каждое — отдельный файл в StackEdit) и создать **итоговый документ** `README.md` для учебного проекта.

Для каждого упражнения ниже дан **конкретный текст для набора** и **указание, как его оформить**. Задача — набрать текст в StackEdit и применить правильную разметку, чтобы в предпросмотре получился нужный результат.

---

5. Ход работы

Шаг 1. Настройка StackEdit

1. Откройте https://stackedit.io/ в браузере.
2. Убедитесь, что предпросмотр отображается справа (если нет — включите его через меню).
3. Создайте новый файл: меню слева → «+» → «New file».
4. Переименуйте файл в `01-basics.md`.

5. Удалите всё содержимое, которое было по умолчанию, — начинаем с чистого листа.

Шаг 2. Упражнение 1. Базовое форматирование

Файл: `01-basics.md`

Наберите в StackEdit следующий текст и примените к нему указанную разметку.

Задание 1.1. Заголовки

Наберите:

```
Моя первая документация
```

Оформите эту строку как **заголовок первого уровня (H1)**.
Как оформить: поставьте перед текстом `# ` (решётка и пробел).

Наберите:

```
Раздел 1. Введение
```

Оформите как **заголовок второго уровня (H2)**.

Наберите:

```
Подраздел 1.1. О чём этот документ
```

Оформите как **заголовок третьего уровня (H3)**.
Как оформить: `### Подраздел 1.1. О чём этот документ`.

> **Правило:** после `#` обязательно ставится **пробел**. Без него заголовок не сработает.

Задание 1.2. Форматирование текста

Наберите предложение:

```
Этот документ создан для изучения формата Markdown.
```

Слово **Markdown** оформите жирным.
Как оформить: `**Markdown**`.

Наберите предложение:

```
Markdown — это лёгкий язык разметки.
```

Слово **лёгкий** оформите курсивом.
Как оформить: `*лёгкий*`.

Наберите предложение:

```
Черновик версии 1.0 больше не актуален.
```

Фразу **версии 1.0** оформите зачёркнутым.
Как оформить: `~~версии 1.0~~`.

Наберите предложение:

```
Команда git status показывает состояние репозитория.
```

Фразу **git status** оформите как код в строке.
Как оформить: `` `git status` `` (обратные кавычки, не обычные!).

Задание 1.3. Цитата и линия

Наберите:

```
Документация — это память проекта.
```

Оформите как цитату.
Как оформить: поставьте `>` в начале строки: `> Документация — это память проекта.`

Ниже цитаты добавьте горизонтальную линию.

> **Важно:** перед `---` и после него должна быть **пустая строка**, иначе линия может не отобразиться или «съесть» предыдущий текст как заголовок.

Шаг 3. Упражнение 2. Списки и ссылки

Файл: `02-lists-links.md` (создайте новый файл в StackEdit)

Задание 2.1. Маркированный список

Наберите список технологий:

```
HTML
CSS
JavaScript
Git
```

Оформите как **маркированный список**.

Задание 2.2. Вложенный список

Наберите:

```
Frontend
HTML
CSS
JavaScript
Backend
PHP
MySQL
```

Оформите как **вложенный маркированный список**: `Frontend` и `Backend` — верхний уровень, остальные — вложенные (с отступом).

Задание 2.3. Нумерованный список

Наберите шаги установки проекта:

```
Открыть терминал
Перейти в папку проекта
Выполнить npm install
Запустить npm run dev
```

Оформите как **нумерованный список**.

Задание 2.4. Чек-лист

Наберите:

```
Свёрстан макет главной страницы
Настроен роутинг
Подключена база данных
```

Оформите как **чек-лист**, где первый пункт — выполнен, остальные — нет.
Как оформить:

```
- [x] Свёрстан макет главной страницы
- [ ] Настроен роутинг
- [ ] Подключена база данных
```

> `[x]` — выполнено, `[ ]` — не выполнено. Пробел внутри скобок обязателен.

Задание 2.5. Ссылки

Наберите:

```
Официальная документация Markdown
```

Оформите как **ссылку** на сайт `https://www.markdownguide.org/`.
Как оформить: `[Официальная документация Markdown](https://www.markdownguide.org/)`.

Наберите:

```
StackEdit
```

Оформите как **ссылку с подсказкой** на `https://stackedit.io/`, подсказка — «Онлайн-редактор Markdown».
Как оформить: `[StackEdit](https://stackedit.io/ "Онлайн-редактор Markdown")`.

Задание 2.6. Изображение

Вставьте **любое изображение** (например, логотип Markdown).
Как оформить: `![Логотип Markdown]`

`(https://upload.wikimedia.org/wikipedia/commons/4/48/Markdown-mark.svg)`.

> **Разница:** ссылка — `[текст](url)`, изображение — `![alt](url)`. Восклицательный знак в начале обязателен.

Шаг 4. Упражнение 3. Таблицы и код

Файл: `03-tables-code.md`

Задание 3.1. Таблица

Создайте таблицу «Технологии проекта» с тремя столбцами: **Технология**, **Назначение**, **Версия**. Заполните четырьмя строками:

Технология	Назначение	Версия
HTML	Разметка страниц	5
CSS	Оформление	3
JavaScript	Интерактивность	ES2023
Git	Контроль версий	2.40

Как оформить:

```
| Технология | Назначение | Версия |
|---|---|---|
| HTML | Разметка страниц | 5 |
| CSS | Оформление | 3 |
| JavaScript | Интерактивность | ES2023 |
| Git | Контроль версий | 2.40 |
```

> **Важно:** вторая строка `|---|---|---|` обязательна — она отделяет заголовок таблицы от данных. Количество столбцов в ней должно совпадать с остальными строками.

Задание 3.2. Выравнивание в таблице

Создайте таблицу «Статус задач» с тремя столбцами: **Задача** (по левому краю), **Приоритет** (по центру), **Срок** (по правому краю). Заполните двумя строками.

Как оформить:

```
| Задача | Приоритет | Срок |
|:---|:---:|---:|
| Вёрстка | Высокий | 15.11 |
| Тесты | Средний | 20.11 |
```

> Двоеточия задают выравнивание: `:---` — влево, `:---:` — по центру, `---:` — вправо.

Задание 3.3. Код в строке и блок кода

Наберите предложение:

```
Для установки зависимостей выполните команду npm install.
```

Команду **npm install** оформите как код в строке.
Как оформить: `` `npm install` ``.

Ниже добавьте **блок кода** с примером HTML:

```
<h1>Заголовок</h1>
<p>Параграф</p>
```

Как оформить: оберните текст в три обратные кавычки с указанием языка:

````
```html
<h1>Заголовок</h1>
<p>Параграф</p>
```
````

> В предпросмотре должна включиться **подсветка синтаксиса** (теги станут цветными).

Задание 3.4. Блок кода без языка

Добавьте блок кода с командами терминала:

```
git clone https://github.com/example/project.git
cd project
npm install
```

Как оформить: оберните в три обратные кавычки **без указания языка**:

````
```
git clone https://github.com/example/project.git
cd project
npm install
```
````

Шаг 5. Упражнение 4. Сложные элементы

Файл: `04-advanced.md`

Задание 4.1. Экранирование

Наберите строку, в которой звёздочки должны отображаться как звёздочки, а не превращаться в курсив:

```
Это *не курсив*, а просто звёздочки.
```

Как оформить: экранируйте звёздочки обратным слэшем: `Это \*не курсив\*, а просто звёздочки.`

Задание 4.2. Комбинированное форматирование

Наберите:

```
Важное замечание
```

Оформите одновременно **жирным и курсивом**.
Как оформить: три звёздочки с каждой стороны: `***Важное замечание***`.

Задание 4.3. Вложенные списки разных типов

Создайте **нумерованный список** из двух пунктов, внутри каждого — **маркированный** подсписок из двух пунктов.

Как оформить:

```
1. Первый шаг
   - подпункт A
   - подпункт B
2. Второй шаг
   - подпункт C
   - подпункт D
```

Задание 4.4. Цитата со списком

Создайте цитату, внутри которой — маркированный список из двух пунктов.
Как оформить: каждую строку цитаты начинайте с `>` :

```
> Основные правила:
> - пишите коротко
> - проверяйте предпросмотр
```

Шаг 6. Итоговый документ `README.md`

Создайте **новый файл** в StackEdit и назовите его `README.md`. Это документация для вашего учебного проекта.

Тему проекта выберите самостоятельно: например, «Сайт кофейни», «Портал онлайн-курсов», «Интернет-магазин», «Личный блог». Далее — заполняйте по шаблону **своим** содержанием, но с обязательным использованием разметки.

Структура документа (обязательные элементы):

1. Заголовок H1

Название проекта.
Пример: `# Сайт кофейни «Утро»`

2. Краткое описание (2–3 предложения)

С элементами форматирования — хотя бы одно слово жирным, одно курсивом.
Пример: `Проект представляет собой **корпоративный сайт** кофейни с меню, фотогалереей и формой обратной связи. Основная цель – *привлечь новых клиентов* и рассказать о преимуществах заведения.`

3. Раздел «Возможности» (H2)

Маркированный список из 4+ пунктов.
Пример:

```
## Возможности

- Просмотр меню с категориями и ценами
- Фотогалерея интерьера
- Форма обратной связи
- Интерактивная карта с адресом
```

4. Раздел «Технологии» (H2)

Таблица минимум с 3 столбцами и 3 строками данных.
Пример:

```
## Технологии

| Технология | Назначение | Версия |
|---|---|---|
| HTML | Разметка | 5 |
| CSS | Стили | 3 |
| JavaScript | Интерактивность | ES2023 |
```

5. Раздел «Установка» (H2)

Нумерованный список шагов + блок кода с командами.
Пример:

````
## Установка

1. Склонируйте репозиторий
2. Перейдите в папку проекта
3. Установите зависимости

```bash
git clone https://github.com/example/coffee-site.git
cd coffee-site
npm install
```
````

```
#### 6. Раздел «Структура проекта» (H2)
```

```
Блок кода с деревом папок.
**Пример:**
```

Структура проекта

```
coffee-site/
├── index.html
├── css/
├── js/
└── images/
```

7. Раздел «Планы» (H2)

Чек-лист минимум из 4 пунктов (1–2 выполненных).
Пример:

```
## Планы

- [x] Свёрстан макет главной страницы
- [x] Подключены стили
- [ ] Добавлена форма обратной связи
- [ ] Настроена карта
```

8. Раздел «Полезные ссылки» (H2)

Минимум 2 ссылки.
Пример:

```
## Полезные ссылки

- [Документация Markdown](https://www.markdownguide.org/)
- [StackEdit](https://stackedit.io/ "Онлайн-редактор")
```

9. Изображение

Логотип или картинка-заглушка.
Пример: `![Логотип проекта](https://via.placeholder.com/150)`

10. Цитата

Девиз или примечание.
Пример: `> «Лучше меньше, да лучше» — принцип этого проекта.`

11. Горизонтальные линии

Между крупными разделами (по желанию).
Как оформить: `---` на отдельной строке с пустыми строками вокруг.

Шаг 7. Проверка и сдача

Проверьте итоговый документ по чек-листу:

- [ ] Заголовок H1 один, остальные — H2 и H3 по иерархии.
- [ ] Есть жирный, курсив, зачёркнутый и код в строке.
- [ ] Есть маркированный, нумерованный и вложенный списки.
- [ ] Есть чек-лист.
- [ ] Есть таблица (минимум 2) с выравниванием.
- [ ] Есть блок кода с указанием языка (подсветка включилась).
- [ ] Есть ссылки (минимум 2) и изображение.
- [ ] Есть цитата и горизонтальные линии.
- [ ] В предпросмотре всё отображается корректно, нет «сырых» символов.

Сдача в StackEdit:

1. Для каждого файла (`01-basics.md` , `02-lists-links.md` , `03-tables-code.md` , `04-advanced.md` , `README.md`) выполните экспорт:
   - меню (три точки вверху справа) → **Export** → **Export as Markdown**;
   - сохраните `.md` -файл на компьютер.
2. Прикрепите все пять `.md` -файлов в задание.
3. **Дополнительно** (по желанию): экспортируйте `README.md` в PDF (Export → Export as PDF) — для наглядности.

Шпаргалка по Markdown

Элемент	Синтаксис	Результат
Заголовок 1	`# Текст`	Крупный заголовок
Заголовок 2	`## Текст`	Подзаголовок
Заголовок 3	`### Текст`	Под-подзаголовок
Жирный	`**текст**`	**текст**
Курсив	`*текст*`	*текст*

Элемент	Синтаксис	Результат
Жирный курсив	`***текст***`	***текст***
Зачёркнутый	`~~текст~~`	~~текст~~
Код в строке	`` `код` ``	`код`
Маркированный список	`- пункт`	• пункт
Нумерованный список	`1. пункт`	1. пункт
Вложенный пункт	`- пункт` (2 пробела)	вложенный
Чек-лист	`- [x] готово` / `- [ ] нет`	☑ / ☐
Ссылка	`[текст](url)`	кликабельный текст
Ссылка с подсказкой	`[текст](url "подсказка")`	с tooltip
Изображение	`![alt](url)`	картинка
Цитата	`> текст`	цитата
Блок кода	```` ```js ... ``` ````	блок с подсветкой
Таблица	`| a | b |` + `|---|---|`	таблица
Выравнивание	`:---`, `:---:`, `---:`	влево, центр, вправо
Линия	`---`	горизонтальная линия
Экранирование	`\*текст\*`	*текст* без курсива

Контрольные вопросы

1. Что такое Markdown? Чем он отличается от HTML и Word?
2. Почему Markdown удобен для Doc as Code?
3. В чём разница между `*текст*` и `**текст**`?
4. Как создать таблицу в Markdown? Как задать выравнивание?
5. Как вставить блок кода с подсветкой синтаксиса? Зачем указывать язык?
6. Чем ссылка отличается от изображения в Markdown?
7. Что такое GitHub Flavored Markdown? Какие расширения он добавляет?
8. Как экранировать спецсимволы? Приведите пример.
9. Зачем экспортировать файлы из StackEdit? Что будет, если этого не сделать?
10. Как вы думаете, почему Markdown стал стандартом для README на GitHub?